

Escuela Politécnica Nacional

Facultad de Ingeniería en Sistemas

INFORME TÉCNICO: IMPLMENTACIÓN DE ANALIZADORES LÉXICOS EN LEX.

Asignatura: Compiladores y Lenguajes

Autores: Aidan Carrasco, Felipe Merino y Mathew Verdezoto

1. Resumen.

El actual informe presenta ejercicios lex desarrollados para la solución de distintos problemas comunes en el área de backend. El análisis léxico constituye a la primera fase de un sistema traductor, el cual transforma un conjunto de flujo continuo de caracteres provenientes de un código fuente en secuencias organizadas con significado semántico llamados tokens.

2. Estructura de un Archivo LEX

Un archivo fuente lex (.l) se encuentra estructurado por los siguientes bloques funcionales separados por ():

- **Área de declaración de variables:** Se declaran las librerías y variables que se van a utilizar.
- **Área de expresiones regulares:** En esta se adjuntan todas las expresiones regulares necesarias para cada programa y su respectiva impresión en consola si se llega a encontrar algo relacionado con la misma.
- **Área de código:** En esta se empieza a codificar en el lenguaje previamente seleccionado (C en este caso).

3. Implementación y Código Fuente

```
%{
/*analizador que  reconoce identificadores, palabras reservadas (if, else,
while, int)
y números enteros. Imprima el tipo de token y su lexema.*/

#include <stdio.h>
%}

%%
"if"|"else"|"while"|"int"    { printf("Token: PALABRA_RESERVADA | Lexema:
```

```

%s\n", yytext); }

[a-zA-Z][a-zA-Z0-9]*      { printf("Token: IDENTIFICADOR | LEXEMA:
%s\n", yytext); }

[0-9]+                    { printf("Token: NUMERO_ENTERO      |
Lexema: %s\n", yytext); }

[ \t\n\r]+               { /* Ignorar los espacios en blanco entre
tokens */ }

.                          { printf("Token: ESTE NO ES UN CARACTER
VALIDO | Lexema: %s\n", yytext); }

%%

int main() {
    printf("Ingrese un codigo de prueba");
    yylex();
    return 0;
}

int yywrap() {
    return 1;
}

```

4. Capturas de Ejecución:

```
acha ~/COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM 02:00
+ cd TRECE/

acha .../COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/TRECE v16.1.1 02:00
+ ls
a.out entrada13.txt lex.yy.c trece.l

acha .../COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/TRECE v16.1.1 02:00
+ ./a.out < entrada13.txt
Ingrese un codigo de pruebaToken: PALABRA_RESERVADA | Lexema: int
Token: IDENTIFICADOR | LEXEMA: main
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: (
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: )
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: {
Token: PALABRA_RESERVADA | Lexema: int
Token: IDENTIFICADOR | LEXEMA: contador
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: =
Token: NUMERO_ENTERO | Lexema: 10
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: ;
Token: PALABRA_RESERVADA | Lexema: while
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: (
Token: IDENTIFICADOR | LEXEMA: contador
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: )
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: {
Token: PALABRA_RESERVADA | Lexema: if
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: (
Token: IDENTIFICADOR | LEXEMA: contador
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: )
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: {
Token: IDENTIFICADOR | LEXEMA: contador
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: =
Token: NUMERO_ENTERO | Lexema: 0
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: ;
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: }
Token: PALABRA_RESERVADA | Lexema: else
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: {
Token: PALABRA_RESERVADA | Lexema: int
Token: IDENTIFICADOR | LEXEMA: auxiliar
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: =
Token: NUMERO_ENTERO | Lexema: 995
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: ;
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: }
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: }
Token: ESTE NO ES UN CARACTER VALIDO | Lexema: }

acha .../COMPID2026A/LAB-LEX-CarrascoA-MerinoA-VerdezotoM/TRECE v16.1.1 02:04
```

5. Conclusión

Se observa que el programa detecta palabras reservadas como if, while, else y les va asignando su lexema a cada uno de ellos. En el caso que no sea un carácter valido va a salir que no es valido.